VECTORS

- It's an array of data elements of the same type. The data elements in a vector are called components. To create a vector, use the function c()

```
> x <- c(3,4,5)
> x
[1] 3 4 5
```

```
> myvec <- c(1,3,1,42)
> myvec
[1]  1  3  1 42
> myvec2 <-c(3,-3,2,5,myvec)
> myvec2
[1]  3 -3  2  5  1  3  1 42
> myvec3 <- c(myvec,myvec2)
> myvec3
 [1]  1  3  1 42  3 -3  2  5  1  3  1 42
```

SEQUENCES, REPETITION, SORTING, AND LENGTHS

Sequences

- creating an equally spaced sequence of increasing or decreasing numeric values

```
> 3:27
 [1]  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27
```

```
> seq(from=3,to=27,by=1)
 [1]  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27
> seq(from=3,to=27,by=3)
[1]  3  6  9 12 15 18 21 24 27
```

SEQUENCES, REPETITION, SORTING, AND LENGTHS

Repetition

```
> rep(x=2,times=5)
[1] 2 2 2 2 2
> rep(x=c(2,20,5,5.3),times=4)
 [1]  2.0 20.0  5.0  5.3  2.0 20.0  5.0  5.3  2.0 20.0  5.0  5.3  2.0 20.0  5.0  5.3
> rep(x=c(2,20,5,5.3),each=2)
[1]  2.0  2.0 20.0 20.0  5.0  5.0  5.3  5.3
> rep(x=c(2,20,5,5.3),times=3,each=2)
 [1]  2.0  2.0 20.0 20.0  5.0  5.0  5.3  5.3  2.0  2.0 20.0 20.0  5.0  5.0  5.3  5.3  2.0  2.0 20.0 20.0  5.0  5.0  5.3
[24]  5.3
```

Sorting

```
> sort(x=c(1.5,-1,-15,4.41),decreasing=FALSE)
[1] -15.00  -1.00   1.50   4.41
> sort(x=c(1.5,-1,-15,4.41),decreasing=TRUE)
[1]   4.41   1.50  -1.00 -15.00
> bar <- sort(x=myvec,decreasing=TRUE)
> bar
[1] 42  3  1  1
```

SEQUENCES, REPETITION, SORTING, AND LENGTHS

Lengths

- determines how many entries exist in a vector given as the argument x

```
> length(x=c(1,3,1,42))
[1] 4
> length(x=myvec)
[1] 4
> length(x=myvec2)
[1] 8
> length(x=myvec3)
[1] 12
```

CLASS EXERCISE

Repeat the vector c(-1,3,-5,7,-9) twice, with each element repeated 10 times, and store the result. Display the result sorted from largest to smallest.

ST331

(Statistical Computing 3)

CLASS EXERCISE

```
A=rep(x=c(-1,3,-5,7,-9),times=2,each=10)
sort(A, decreasing=TRUE)
 [1]  7  7  7  7  7  7  7  7  7  7  7  7  7  7  7  7  7  7  7  3  3  3  3  3  3  3  3  3  3  3  3  3  3
[37]  3  3  3  3 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -5 -5 -5 -5 -5 -5 -5 -5 -5 -5 -5
[73] -5 -5 -5 -5 -5 -5 -5 -5 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9 -9
```

ST331

(Statistical Computing 3)

SUBSETTING AND ELEMENT EXTRACTION

Subsetting

- allows you to retrieve specific elements from a vector

```
> myvec3<-c(1,3,1,42,3,-3,2,5,1,3,1,42)
> myvec[2]
[1] 3
> myvec3[length(x=myvec3)]
[1] 42
> myvec3[-1]
 [1]  3  1 42  3 -3  2  5  1  3  1 42
```

Vector operations

```
> breadth<-c(5.5,4.5,3.5,2.5,1.5,0.5)
> Height<-c(1,-1)
> breadth*Height
[1]   5.5 -4.5   3.5 -2.5   1.5 -0.5
```

IMPORTANT CODES

Function/operator	Brief description
+, *, -, /, ^	Arithmetic
sqrt	Square root
log	Logarithm
exp	Exponential
<-, =	Object assignment
c	Vector creation
:, seq	Sequence creation
rep	Value/vector repetition
sort	Vector sorting
length	Determine vector length
[ ]	Vector subsetting/extraction
sum	Sum all vector elements
prod	Multiply all vector elements

```
> sum(breadth)
[1] 18
> prod(breadth)
[1] 162.4219
```

MATRICES

- A matrix is a 2-dimensional array of data elements of the same type. In statistics, matrices are useful as tables that hold data.

```
> matrix(data=c(1,2,3,4,5,6),nrow=2,ncol=3,byrow=FALSE)
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
> matrix(data=c(1,2,3,4,5,6),nrow=2,ncol=3,byrow=TRUE)
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
> rbind(1:3,4:6)
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
> cbind(1:3,4:6)
     [,1] [,2]
[1,]    1    4
[2,]    2    5
[3,]    3    6
```

MATRIX DIMENSIONS

- Another useful function, dim, provides the dimensions of a matrix stored in your workspace.

```
> dim(A)
[1] 2 2
> nrow(A)
[1] 2
> ncol(A)
[1] 2
```

SUBSETTING

- Extracting and subsetting elements from matrices in R is much like extracting elements from vectors. The only complication is that you now have an additional dimension.

```
> B <- matrix(c(0.3,4.5,55.3,91,0.1,105.5,-4.2,8.2,27.9),nrow=3,ncol=3)
> B
     [,1]  [,2] [,3]
[1,]  0.3  91.0 -4.2
[2,]  4.5   0.1  8.2
[3,] 55.3 105.5 27.9
> B[3,2]
[1] 105.5
```

ROW, COLUMN, AND DIAGONAL EXTRACTIONS

- To extract an entire row or column from a matrix, you simply specify the desired row or column number and leave the other value blank

```
> B[,2]
[1]  91.0   0.1 105.5
> B[1,]
[1]  0.3 91.0 -4.2
```

```
> B[2:3,]
     [,1]  [,2] [,3]
[1,]  4.5   0.1  8.2
[2,] 55.3 105.5 27.9
> B[,c(3,1)]
     [,1] [,2]
[1,] -4.2  0.3
[2,]  8.2  4.5
[3,] 27.9 55.3
> B[c(3,1),2:3]
      [,1] [,2]
[1,] 105.5 27.9
[2,]  91.0 -4.2
> diag(x=B)
[1]  0.3  0.1 27.9
```

ROW, COLUMN, AND DIAGONAL EXTRACTIONS

- To extract an entire row or column from a matrix, you simply specify the desired row or column number and leave the other value blank

```
> B[,2]
[1]  91.0   0.1 105.5
> B[1,]
[1]  0.3 91.0 -4.2
```

```
> B[2:3,]
     [,1]  [,2] [,3]
[1,]  4.5   0.1  8.2
[2,] 55.3 105.5 27.9
> B[,c(3,1)]
     [,1] [,2]
[1,] -4.2  0.3
[2,]  8.2  4.5
[3,] 27.9 55.3
> B[c(3,1),2:3]
      [,1] [,2]
[1,] 105.5 27.9
[2,]  91.0 -4.2
> diag(x=B)
[1]  0.3  0.1 27.9
```

CLASS EXERCISE

Construct and store a 4 x 2 matrix that's filled row-wise with the values 4.3, 3.1, 8.2, 8.2, 3.2, 0.9, 1.6, and 6.5, in that order.

ST331

(Statistical Computing 3)

CLASS EXERCISE

```
> matrix(data=c(4.3, 3.1, 8.2, 8.2, 3.2, 0.9, 1.6, 6.5),nrow=4,ncol=2,byrow=TRUE)
     [,1] [,2]
[1,]  4.3  3.1
[2,]  8.2  8.2
[3,]  3.2  0.9
[4,]  1.6  6.5
```

ST331

(Statistical Computing 3)

MATRIX OPERATIONS AND ALGEBRA

Matrix Transpose

```
> A <- rbind(c(2,5,2),c(6,1,4))
> A
     [,1] [,2] [,3]
[1,]    2    5    2
[2,]    6    1    4
> t(A)
     [,1] [,2]
[1,]    2    6
[2,]    5    1
[3,]    2    4
> t(t(A))
     [,1] [,2] [,3]
[1,]    2    5    2
[2,]    6    1    4
```

Identity Matrix

```
> A <- diag(x=3)
> A
     [,1] [,2] [,3]
[1,]    1    0    0
[2,]    0    1    0
[3,]    0    0    1
```

MATRIX OPERATIONS AND ALGEBRA

Scalar Multiple of a Matrix

```
> A <- rbind(c(2,5,2),c(6,1,4))
> a <- 2
> a*A
     [,1] [,2] [,3]
[1,]    4   10    4
[2,]   12    2    8
```

MATRIX OPERATIONS AND ALGEBRA

Matrix Addition and Subtraction

```
> A <- cbind(c(2,5,2),c(6,1,4))
> A
     [,1] [,2]
[1,]    2    6
[2,]    5    1
[3,]    2    4
> B <- cbind(c(-2,3,6),c(8.1,8.2,-9.8))
> B
     [,1] [,2]
[1,]   -2  8.1
[2,]    3  8.2
[3,]    6 -9.8
> A-B
     [,1] [,2]
[1,]    4 -2.1
[2,]    2 -7.2
[3,]   -4 13.8
> A+B
     [,1] [,2]
[1,]    0 14.1
[2,]    8  9.2
[3,]    8 -5.8
```

MATRIX OPERATIONS AND ALGEBRA

Matrix Multiplication

```
> A <- rbind(c(2,5,2),c(6,1,4))
> dim(A)
[1] 2 3
> B <- cbind(c(3,-1,1),c(-3,1,5))
> dim(B)
[1] 3 2
> A%*%B
     [,1] [,2]
[1,]    3    9
[2,]   21    3
```

Matrix Inversion

```
> A <- matrix(data=c(3,4,1,2),nrow=2,ncol=2)
> A
     [,1] [,2]
[1,]    3    1
[2,]    4    2
> solve(A)
     [,1] [,2]
[1,]    1 -0.5
[2,]   -2  1.5
> A%*%solve(A)
     [,1] [,2]
[1,]    1    0
[2,]    0    1
```

CLASS EXERCISE

Calculate the following:

$$\frac{2}{7}\left(\begin{bmatrix}1 & 2\\ 2 & 4\\ 7 & 6\end{bmatrix} - \begin{bmatrix}10 & 20\\ 30 & 40\\ 50 & 60\end{bmatrix}\right)$$

ST331

(Statistical Computing 3)

IMPORTANT CODES

Function/operator	Brief description
`matrix`	Create a matrix
`rbind`	Create a matrix (bind rows)
`cbind`	Create a matrix (bind columns)
`dim`	Get matrix dimensions
`nrow`	Get number of rows
`ncol`	Get number of columns
`[ , ]`	Matrix/array subsetting
`diag`	Diagonal elements/identity matrix
`t`	Matrix transpose
`*`	Scalar matrix multiple
`+, -`	Matrix addition/subtraction
`%*%`	Matrix multiplication
`solve`	Matrix inversion